
SMYRF: Efficient Attention using Asymmetric Clustering

Giannis Daras

Computer Science Department
The University of Texas at Austin
giannisdaras@utexas.edu

Augustus Odena

Google Research
augustusodena@google.com

Nikita Kitaev

Google Research
kitaev@cs.berkeley.edu

Alexandros G. Dimakis

ECE Department
The University of Texas at Austin
dimakis@austin.utexas.edu

Abstract

We propose a novel type of balanced clustering algorithm to approximate attention. Attention complexity is reduced from $O(N^2)$ to $O(N \log N)$, where N is the sequence length. Our algorithm, SMYRF, uses Locality Sensitive Hashing (LSH) in a novel way by defining new Asymmetric transformations and an adaptive scheme that produces balanced clusters. The biggest advantage of SMYRF is that it can be used as a drop-in replacement for dense attention layers *without any retraining*. On the contrary, prior fast attention methods impose constraints (e.g. queries and keys share the same vector representations) and require re-training from scratch. We apply our method to pre-trained state-of-the-art Natural Language Processing and Computer Vision models and we report significant memory and speed benefits. Notably, SMYRF-BERT outperforms (slightly) BERT on GLUE, while using 50% less memory. We also show that SMYRF can be used interchangeably with dense attention before and after training. Finally, we use SMYRF to train GANs with attention in high resolutions. Using a single TPU, we were able to scale attention to $128 \times 128 = 16k$ and $256 \times 256 = 65k$ tokens on BigGAN on CelebA-HQ.

1 Introduction

Attention layers enable long-range representation learning and are becoming indispensable in architectures for both Image Synthesis [1, 2, 3] and Natural Language Processing [4, 5, 6, 7, 8, 9]. Attention finds further uses in other domains like symbolic mathematics and music modeling as well [10, 11, 12]. Unfortunately, attention layers have high computational and memory cost which scales quadratically in the size of the input sequence. This constraint is so onerous that the canonical implementation of attention for image synthesis - Self-Attention GAN [2] - could only afford to use one self-attention layer. For NLP, modern transformer-based models can only be trained in large industry research labs with massive infrastructure investments. For instance, the recently published GPT-3 [13] model uses 96 attention layers trained on input sequences of 2048 tokens. When fine-tuning pre-trained attention models, NLP researchers usually truncate input sentences, limiting performance on datasets with longer inputs.

Recent research [14, 3] indicates that dense attention is statistically and computationally inefficient [15, 16, 3]: it does not account for the locality inherent in many tasks. Alternatives have been proposed that are either more efficient [12, 17, 18, 19, 20, 7, 21, 22] or that better accommodate locality [23, 3]. Most such alternatives have been sparse. Sparsity can be achieved by limiting

attention to pre-defined positions [23, 3, 22, 12]. Recent work [17, 18, 19, 20] proposes data-driven sparsity, which allows for discovery of arbitrarily complex dependencies between input positions.

Despite this progress, new state-of-the-art models [8, 13, 9, 24, 25, 26] still use the original dense attention layers. There are three reasons for this: (i) alternative fast-attention mechanisms degrade the performance of the underlying model. For example, replacing dense attention layers in Transformers with memory efficient local attention [23] increases perplexity from 41.57 to 44.23 [20]. (ii) some mechanisms work well, but make very strict assumptions. For example, in Star Transformer [22] all nodes attend to a relay node which summarizes the content of the entire input sequence, but this prevents the use of causal masking, so it can only be used for encoding. (iii) some alternatives are only efficient in theory. For example, in some variants [17, 27] sparsification of the attention map happens after instantiating the matrix, and so quadratic memory is still used before instantiation. Finally, [12, 28] require highly specialized GPU-kernels and which prevents usage in several hardware settings (e.g. TPUs). The design of fast and efficient attention layers remains a challenge.

Our Contributions:

- 1) We propose a novel type of balanced clustering to approximate attention. We call the underlying optimization problem Attention Biclustering and prove that finding an exact solution is computationally intractable.
- 2) We propose an algorithm for solving Attention Biclustering efficiently in practice. Our algorithm, SMYRF, uses Locality Sensitive Hashing (LSH) in a novel way by defining new Asymmetric transformations and an adaptive scheme that produces balanced clusters.
- 3) Our method, SMYRF, can handle different query and key vectors, just like normal dense attention. As a result, SMYRF layers are drop-in replacements for pre-trained models, unlike previously proposed fast-attention mechanisms such as Sinkhorn [20], Reformer [18] and Routing Transformer [19].
- 4) We show through numerous experiments that SMYRF attention layers are very effective in terms of performance, memory and speed, even without any training. We measure the memory-performance trade-off of applying SMYRF to state-of-the-art NLP and Computer Vision models, across more than a dozen tasks. For example, we are able to shrink the memory requirements of a pre-trained BigGAN [1] by 50% while maintaining 98.2% of its Inception score without re-training.
- 5) We finetune SMYRF on GLUE [25] starting from a BERT (base) checkpoint. We demonstrate that SMYRF-BERT outperforms BERT while using 50% less memory. We also show that with 75% less memory, SMYRF maintains 99% of BERT performance on GLUE. Due to SMYRF’s portability, we are also able to conduct experiments for various memory configurations with pre-trained BERT and RoBERTa [9] models on IMDB. We show slight performance drops for great memory benefits.
- 6) We show that SMYRF can be interchanged with dense layers *before* and *after* training. We report performance gains by using SMYRF in a back-and-forth manner: we replace dense with SMYRF during training (to earn in memory) and we replace SMYRF with dense attention during inference (to earn in performance). The interchangeability of SMYRF with dense attention is unique, as it has not been observed in previously proposed attention alternatives [18, 19, 20, 28, 3].
- 7) We are able to scale the resolution of attention for GANs, due to our reduced memory footprint. We train a BigGAN with an 128×128 SMYRF attention layer and show it outperforms the dense attention performance, decreasing FID from 26.06 to 25.03 in Celeba-HQ-128 [29]. Finally, we successfully train a BigGAN with attention at resolution 256×256 on a single v3-8 TPU.
- 8) We open-source our code and pre-trained models to encourage more related research: <https://github.com/giannisdaras/smyrf>.

2 Background

Attention [30] works by computing inner products of query and key vectors. Depending on the application, these vectors may represent embeddings for tokens or image pixels. Input of each attention layer is three sets: $\mathcal{Q}, \mathcal{K}, \mathcal{V}$ for query, key and value vectors respectively. Attention of q to the keys set $\mathcal{K}$ outputs a new vector o_q , which is a weighted sum of value vectors $v_i \in \mathcal{V}$ where each weight w_i increases with the inner product $q \cdot k_i$. Specifically, the output is computed as:

$$o_q = \sum_{i=1}^N w_i v_i, \quad w_i = \frac{e^{q \cdot k_i}}{\sum_{j=1}^N e^{q \cdot k_j}}. \quad (1)$$

Here, we assumed for notational simplicity that $N = |\mathcal{Q}| = |\mathcal{K}|$. Using matrix notation, attention is equivalently defined as $\sigma(Q \cdot K^T) \cdot V$ where Q, K, V are matrices with rows the embeddings for each query, key, value and the function $\sigma(\cdot)$ computes the row-wise softmax.

3 Approximating Attention with Clustering

3.1 Motivation

Our method is motivated by the observation that attention matrices have interesting structure in real datasets. Naively, to compute dense attention, as equation 1 shows, we need to compute all outputs o_{q_i} , i.e. $O(|\mathcal{Q}| \cdot |\mathcal{K}|)$, a quadratic number of inner products $q_i \cdot k_j$, $q_i \in \mathcal{Q}$, $k_j \in \mathcal{K}$. However, we observe that in most real networks, the attention weights w_i are sparse, because of the softmax operation and the structure of the vectors. For example we observe that in a pre-trained BigGAN on ImageNet, on average $98.11 \pm 0.26\%$ ¹ of keys get weight less than 0.01 in softmax and $86.11 \pm 2.92\%$ of them get less than $\frac{1}{|\mathcal{K}|}$, where $\mathcal{K}$ is the number of keys.

Further, we observe that the attention matrix is near low-rank, even after the softmax. By definition, the matrix $Q \cdot K^T$ is going to be of rank at most the dimension of the query and key vectors. Therefore, if the embeddings dimension is smaller than the input sequence, the attention matrix is low-rank. This is more pronounced for images and long-context language models. However, one can easily construct cases of low-rank matrices which become full rank after softmax. Our finding is that this does not happen in practice. In the Appendix we show that *real attention matrices of pretrained models have a sharp decay in their singular values and hence can be well approximated by low-rank matrices*.

SMYRF benefits from sparsity and low-rank structure of attention matrices. By clustering keys and queries into groups, we obtain block-diagonal structure in the approximate attention matrix, since only query-key pairs within the same cluster are computed. We show that this method leads to accurate approximations of dense attention and it can be computed much faster and with much less memory.

3.2 Problem Formulation

We formulate the assignment of keys and queries into clusters as an optimization problem. Denote with $P_{ij} = q_i^T k_j$ the element (i, j) of the product matrix $P = Q \cdot K^T$ and the attention map with $M = \sigma(Q \cdot K^T)$. We will assign query and key vectors into L clusters $c_1, c_2, \dots, c_L$ and compute attention only within each cluster. For fast execution on TPUs/GPUs, all partial attentions should be computed in parallel. For this reason, we require that clusters are balanced: i.e. all clusters contain the same number of keys and queries. We note that the number of keys in each cluster does not have to be equal to the number of queries. Formally, each cluster contains $\frac{|\mathcal{Q}|}{L}$ queries and $\frac{|\mathcal{K}|}{L}$ keys.

We denote with $\mathcal{C}^L$ the set of all possible assignments in L balanced non-overlapping clusters. A specific assignment is denoted by $\mathcal{C}_t^L$ and there are T possible such assignments, where T is exponentially large in the number of keys and queries.

$$\mathcal{C}^L = \{\mathcal{C}_1^L, \mathcal{C}_2^L, \dots, \mathcal{C}_T^L\}.$$

$$\mathcal{C}_t^L = \{c_1, c_2, \dots, c_L\} : \begin{cases} c_i = \{q_1, \dots, q_{\frac{|\mathcal{Q}|}{L}}, k_1, \dots, k_{\frac{|\mathcal{K}|}{L}}\}, & c_i \subseteq \mathcal{Q} \cup \mathcal{K}, \forall i \in \{1, \dots, L\} \\ c_x \cap c_y = \emptyset & \forall c_x, c_y \in \mathcal{C}_t^L. \end{cases} \quad (2)$$

We emphasize that every key and query is assigned in a unique cluster for any valid assignment $\mathcal{C}_t^L$: $c_x \cap c_y = \emptyset \quad \forall c_x, c_y \in \mathcal{C}_t^L$. We also define a masking operator Mask_ϵ that takes as input: (i) a clustering $\mathcal{C}_t^L \in \mathcal{C}^L$ and (ii) the product matrix P and replaces (q, k) pairs that are not in the same cluster with $-a$, where $a \in \mathbb{R}^+$ is a constant chosen to satisfy $e^{-a} = \epsilon$ for a given $\epsilon \geq 0$. Formally:

$$\text{Mask}_\epsilon(\mathcal{C}_t^L, P_{ij}) = \begin{cases} P_{ij} & \text{iff } \exists t : (i, j) \in c_t, \\ -a, & \text{o/w.} \end{cases}$$

¹The reported numbers are calculated by inspecting the attention maps of 1000 random generated images.

Intuitively, the masking operator replaces inner products of queries and keys that are not in the same cluster with an arbitrarily small number, so that the softmax will assign a score arbitrarily close to zero to these entries. We denote with $\hat{P}_\epsilon = \text{Mask}_\epsilon(\mathcal{C}_t^L, P)$ the product matrix after the masking. With this notation, $\hat{P}_0 = \text{Mask}_0(\mathcal{C}_t^L, P)$, is the product matrix for the within-clusters attention.

Attention Biclustering: Under this formulation, we are searching for the cluster assignment $\mathcal{C}_t^L$ that approximates the dense attention matrix $\sigma(P)$ as well as possible, in Frobenius norm:

$$\min_{\mathcal{C}_t^L \in \mathcal{C}^L} \|\sigma(\hat{P}_0) - \sigma(P)\|_F. \quad (3)$$

Note that L must divide the number of queries and keys for this problem to be well-defined.

3.3 Complexity of Attention Biclustering

We start by showing that Attention Biclustering, the optimization problem defined in (3), is provably computationally intractable.

Theorem 1. *Attention Biclustering (3) is NP-hard.*

We defer the proof of this theorem to the Appendix. Our proof proceeds by first establishing hardness before the softmax, using a reduction from three dimensional matching [31]. We then leverage this to establish hardness of approximating attention through clustering after the softmax operation.

We consider it interesting to establish the computational intractability of Attention Biclustering, since this clustering formulation is quite unique due to the softmax operation. Our hardness result rules out an exact polynomial solution, unless $P=NP$. We propose an efficient algorithm that leverages hashing to assign queries and keys to clusters. Formally proving an approximation guarantee or provable inapproximability for the attention approximation problem we proposed remains open.

3.4 Proposed algorithm: SMYRF

Our algorithm consists of the following steps:

- 1) We first propose novel asymmetric transformations $F, G : \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$ such that for all given queries $q_1, q_2 \in \mathcal{Q}$ and keys $k \in \mathcal{K}$: $q_1 \cdot k \leq q_2 \cdot k \iff \|F(q_1) - G(k)\|_2 \leq \|F(q_2) - G(k)\|_2$.
- 2) We then use a Locality Sensitive Hashing (LSH) function $h : \mathbb{R}^{d'} \rightarrow \mathbb{R}$ to map transformed vectors in real numbers, so that that vectors that are close in Euclidean distance correspond to numbers that are close on the real line.
- 3) We sort vectors based on their LSH value and group them by adapting the thresholds to ensure L balanced clusters.
- 4) We perform dense attention within each cluster.

Our approximate attention algorithm relies on a few technical innovations:

Novel Asymmetric Transformations: We need an efficient way to find, for any given query vector $q_i \in \mathcal{Q}$ the set of keys with which it has big inner products. This problem, called Maximum Inner Product Search (MIPS), can be efficiently solved by transforming query and key vectors to convert it to a Nearest Neighbor Search (NNS) as proposed in the pioneering Asymmetric LSH (Locality Sensitive Hashing) work by Shrivastava et al. [32].

We are looking for functions $F : \mathbb{R}^d \rightarrow \mathbb{R}^{d'}, G : \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$ such as: $\|F(q) - G(k)\|_2^2 = D(q \cdot k), \forall (q, k)$ where $D : \mathbb{R} \rightarrow \mathbb{R}$ a decreasing function that depends only on the inner product $q \cdot k$. We constrain our focus on functions D that decrease linearly with the inner product $q \cdot k$. Several previous works have proposed Asymmetric LSH transformations [32, 33, 34] but focus on the case where we have a *single query* q and multiple keys. In that case, any norm $\|q\|_a$ where $a = \{1, \dots, \infty\}$ is constant and thus $D = D(q \cdot k, \|q\|_a)$.

Our central algorithmic contribution is the proposal of novel asymmetric functions:

$$F(q_i) = \left[q_i; 0; \sqrt{M_Q^2 + M_K^2 - \|q_i\|_2^2} \right], \quad G(k_i) = \left[k_i; \sqrt{M_Q^2 + M_K^2 - \|k_i\|_2^2}; 0 \right] \quad (4)$$

where we use the constants $M_Q = \max_{q_i} \|q_i\|_2$, $M_K = \max_{k_i} \|k_i\|_2$, or any other upper bound on the norms. With this transformation, all queries and keys are mapped to a $(d+2)$ -dimensional